

Smart Travel Planner (Due date: 15/6/26)

Frontend and Database need to choose what to use for implementation

Frontend: React / HTML / CSS / JS

Backend: Node.js and Express (Fixed)

Database: MongoDB

Ideas for the functions and features (If got any ideas please add inside or remove the anything can also)

Core CRUD Features (Self-Developed API)

- **Trip Management:** Create, view, update, and delete travel itineraries.
- **Destinations & Notes:** Allow users to save specific destinations with personal notes and categories (e.g., Business, Vacation, Foodie).
- **Preferences:** A "User Preferences" endpoint to store data like dietary restrictions or preferred weather types to filter third-party results later.

Advanced Security Features

- **JWT Authentication:** Use **JSON Web Tokens** for secure session management.
- **Password Hashing:** Use libraries like **bcrypt** to ensure passwords are never stored in plaintext.
- **Role-Based Access Control (RBAC):** Differentiate between **"Admin"** (can manage all data) and **"User"** (can only manage their own trips).

Smart Integration

1. **Weather-Adjusted Packing List:** Fetch the **OpenWeatherMap API** for a saved trip destination and generate a suggested packing list based on the forecast.
2. **Nearby Discovery:** Use **Google Places** or **Foursquare** to find "Attractions" that match the user's stored "Preferences" (e.g., if a user likes "History," show museums near their saved hotel).
3. **Smart Time-to-Visit:** Compare a user's planned visit time with real-time data to warn them if a location is currently closed or experiencing bad weather.
4. **Smart Calendar:** An overview of the trip's duration and daily breakdown
5. **Smart group:** Enables real-time collaborative editing. Like I invite u to join my group so we can view the plan together?

Actual features and functions (if got any ideas please remove or add inside)

1. Authentication & User Management

- **Sign Up:** User registration with password hashing (e.g., using `bcrypt`).
- **Login:** Secure authentication using JWT (`JSON Web Tokens`).
- **Profile Management:** A `GET /api/user/profile` endpoint to retrieve and update user travel preferences (e.g., preferred activities or dietary needs).

2. Self-Developed API (The "Travel Journal")

CRUD operations for managing user travel data:

- **Create Trip (`POST /api/trips`):** Save a new trip including destination, start/end dates, and a travel category.
- **Read Trips (`GET /api/trips`):** Retrieve a list of all saved trips for the authenticated user.
- **Read Single Trip (`GET /api/trips/{id}`):** Retrieve specific details and notes for a single trip.
- **Update Trip (`PUT /api/trips/{id}`):** Edit trip details or add personal travel notes.
- **Delete Trip (`DELETE /api/trips/{id}`):** Remove a trip from the database.

This can choose what to include:

3. Third-Party Integration ("Smart" Features)

- **Live Weather Sync:** Automatically fetch current weather or forecasts for a saved trip destination using the `OpenWeatherMap API`.
- **Nearby Attractions:** Use the `Google Places` or `Foursquare API` to suggest spots near the user's saved destination.
- **Search/Discovery:** A function allowing users to input a location to see both real-time external data and any personal notes they previously saved for that area.
- **Invite People:** Enables real-time collaborative editing via shared links
- **A Calendar:** An overview of the trip's duration and daily breakdown

Task Separations

Database (together?)

1. Frontend:
2. Backend (Authentication & User Management):
3. Backend (CRUD):
4. Backend (Third-Party Integration):

Report (maximum 2000 words)

System architecture

API description

Test results/screenshots

Challenges faced

Submission Deliverables (Due date: 15/6/26)

1. **Source Code:** All code must be hosted on **GitHub** with a clear history of branches, commits, and pull requests.
2. **Report (PDF):** A maximum 2000-word document covering system architecture, API descriptions, test results/screenshots, and challenges faced.
3. **Video:** A YouTube video demonstrating and explaining the system's functionalities.
4. **Demonstration:** A face-to-face presentation where all group members must be present to answer questions.
